

STREAMING EVENT ANOMALY DETECTOR WITH AI AGENT

FINAL PROJECT REPORT

ABSTRACT

The Streaming Event Anomaly Detector is an AI-powered real-time monitoring platform designed to process continuous event streams, identify anomalous behavior, provide intelligent explanations, and trigger automated alerts. The system combines statistical anomaly detection techniques with Large Language Models (LLMs) to improve operational awareness and accelerate incident response.

The platform continuously receives events, analyzes them using a Z-Score-based anomaly detection engine, generates AI-powered explanations using Llama 3.1 through Ollama, and performs autonomous decision-making through an AI Agent. Critical anomalies are automatically communicated to stakeholders through Discord notifications while all system activities are visualized through a centralized monitoring dashboard.

The solution demonstrates how Artificial Intelligence can enhance traditional monitoring systems by moving beyond anomaly identification and providing contextual explanations, severity assessments, and recommended actions.

Keywords: Anomaly Detection, Artificial Intelligence, Event Streaming, FastAPI, React, Ollama, Llama 3.1, Monitoring Dashboard, Discord Alerts.

CHAPTER 1

INTRODUCTION

Background

Modern applications generate massive quantities of operational events every second. These events contain valuable information about system health, performance, user activity, and security status.

Manual monitoring becomes increasingly difficult as event volumes grow. Traditional monitoring systems can identify anomalies but often fail to explain why an anomaly occurred or what corrective action should be taken.

Artificial Intelligence provides an opportunity to improve monitoring systems by generating contextual explanations and recommendations that assist operational teams.

Problem Statement

Organizations face challenges in identifying abnormal patterns within high-volume event streams. Delayed anomaly detection can result in performance degradation, service outages, security incidents, and operational inefficiencies.

An automated system is required to continuously monitor event streams, detect anomalies in real time, provide meaningful explanations, and notify stakeholders immediately.

Proposed Solution

The Streaming Event Anomaly Detector combines statistical anomaly detection with AI-powered analysis to create an intelligent monitoring platform capable of:

- Real-time event processing
- Automated anomaly detection
- AI-generated explanations
- Severity classification
- Automated alerting
- Dashboard-based monitoring

CHAPTER 2

OBJECTIVES

Primary Objectives

- Monitor streaming events continuously.
- Detect abnormal patterns automatically.
- Generate AI-powered explanations.
- Classify anomaly severity.
- Trigger real-time notifications.
- Store historical monitoring data.
- Provide centralized monitoring dashboards.

Secondary Objectives

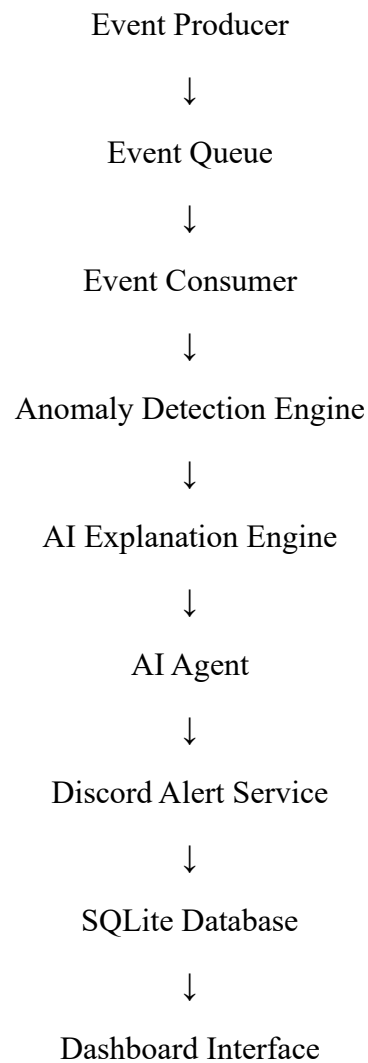
- Improve incident response time.
- Reduce manual analysis effort.
- Enhance operational visibility.

- Demonstrate AI-assisted monitoring techniques.

CHAPTER 3

SYSTEM ARCHITECTURE

Architecture Overview



Component Description

Event Producer

- Generates simulated real-time events.

Event Queue

- Temporarily stores incoming events before processing.

Event Consumer

- Processes queued events sequentially.

Anomaly Detection Engine

- Calculates statistical deviations using Z-Score analysis.

AI Explanation Engine

- Uses Llama 3.1 through Ollama to generate natural language explanations.

AI Agent

- Performs decision-making and recommends corrective actions.

Alert Service

- Delivers critical notifications through Discord webhooks.

Dashboard

- Displays metrics, anomalies, and alerts visually.

CHAPTER 4

TECHNOLOGY STACK

Frontend Technologies

- React
- Vite
- JavaScript
- Recharts

Backend Technologies

- FastAPI
- Python

Database

- SQLite

Artificial Intelligence

- Ollama
- Llama 3.1

Security

- JWT Authentication
- Role-Based Access Control (RBAC)

Notification System

- Discord Webhooks

Containerization

- Docker

CHAPTER 5

SYSTEM MODULES

Module 1: Authentication System

Responsibilities:

- User Registration
- User Login
- JWT Token Generation
- Access Control

Features:

- Secure Authentication
- Protected APIs
- Role-Based Permissions

Module 2: Event Generation Module

Responsibilities:

- Generate streaming events
- Simulate real-world event flows
- Produce test datasets

Features:

- Continuous Event Generation
- Dynamic Event Values

Module 3: Event Processing Module

Responsibilities:

- Receive events
- Process event streams
- Forward data to detection engine

Features:

- Queue-Based Processing
- Real-Time Event Handling

Module 4: Anomaly Detection Engine

Responsibilities:

- Analyze event values
- Identify abnormal behavior

Technique Used:

Z-Score Analysis

Formula:

$$Z = (X - \mu) / \sigma$$

Where:

X = Event Value

μ = Mean Value

σ = Standard Deviation

Features:

- Statistical Detection
- Threshold-Based Classification

Module 5: AI Explanation Engine

Responsibilities:

- Explain anomalies
- Provide contextual analysis

Technology:

Llama 3.1 via Ollama

Example Output:

"The event value significantly exceeds the normal operating range and may indicate abnormal system behavior."

Module 6: AI Agent

Responsibilities:

- Severity Classification
- Root Cause Analysis
- Recommendation Generation
- Alert Decision Making

Severity Levels:

- LOW
- MEDIUM
- HIGH
- CRITICAL

Module 7: Alert Management

Responsibilities:

- Deliver notifications
- Inform operational teams

Technology:

Discord Webhooks

Features:

- Instant Alerts
- Critical Event Notifications

Module 8: Dashboard Monitoring

Responsibilities:

- Visualize metrics

- Display anomalies
- Show alert history

Features:

- KPI Cards
- Charts
- Tables
- Real-Time Updates

CHAPTER 6

DATABASE DESIGN

Users Table

Stores user information and authentication data.

Fields:

- id
- username
- password_hash
- role
- created_at

Events Table

Stores incoming event records.

Fields:

- id
- timestamp
- source
- value
- status

Anomalies Table

Stores anomaly detection results.

Fields:

- id
- event_id
- z_score
- severity
- explanation

Agent Decisions Table

Stores AI-generated recommendations.

Fields:

- id
- anomaly_id
- recommendation
- decision

CHAPTER 7

IMPLEMENTATION METHODOLOGY

Step 1

- Setup FastAPI Backend

Step 2

- Configure SQLite Database

Step 3

- Implement JWT Authentication

Step 4

- Develop Event Generator

Step 5

- Create Queue-Based Processing

Step 6

- Implement Z-Score Detection

Step 7

- Integrate Ollama

Step 8

- Develop AI Agent

Step 9

- Configure Discord Alerts

Step 10

- Build React Dashboard

Step 11

- Perform Testing

Step 12

- Containerize with Docker

CHAPTER 8

TESTING AND VALIDATION

Functional Testing

Verified:

- Authentication
- Event Processing
- Anomaly Detection
- Dashboard Operations

Security Testing

Verified:

- JWT Authentication
- Password Protection
- Access Restrictions

Performance Testing

Verified:

- Continuous Event Processing
- Dashboard Responsiveness
- Alert Delivery

AI Validation

Verified:

- Explanation Generation
- Severity Classification
- Recommendation Accuracy

CHAPTER 9

RESULTS

The developed platform successfully demonstrated:

- ✓ Real-Time Event Processing
- ✓ Statistical Anomaly Detection
- ✓ AI-Powered Explanations
- ✓ AI Agent Decision Making
- ✓ Automated Discord Alerts
- ✓ Dashboard Monitoring
- ✓ Secure Authentication
- ✓ Historical Data Storage

The system accurately detected abnormal events and generated meaningful explanations that improved interpretability and operational awareness.

CHAPTER 10

ADVANTAGES

- Real-Time Monitoring
- Automated Detection
- Intelligent Explanations
- Faster Incident Response
- Improved Operational Visibility
- Automated Notifications
- Easy Deployment

- Scalable Architecture

CHAPTER 11

LIMITATIONS

- Uses SQLite instead of enterprise databases.
- Uses simulated event streams.
- Relies on statistical thresholds.
- Limited horizontal scalability.
- In-memory queue instead of Kafka.

CHAPTER 12

FUTURE ENHANCEMENTS

Kafka Integration

Replace in-memory queue with Apache Kafka.

PostgreSQL Migration

Improve scalability and performance.

Machine Learning Models

Implement:

- Isolation Forest
- Autoencoders
- Time-Series Forecasting

Cloud Deployment

Deploy using:

- AWS
- Azure
- Google Cloud

Kubernetes Support

Enable container orchestration.

Multi-Tenant Architecture

Support multiple organizations.

CHAPTER 13

CONCLUSION

The Streaming Event Anomaly Detector successfully demonstrates the integration of anomaly detection, Artificial Intelligence, real-time monitoring, and automated alerting within a single platform.

The system not only detects abnormal behavior but also provides intelligent explanations and actionable recommendations through an AI Agent. This significantly improves monitoring effectiveness and reduces the effort required for manual investigation.

The project establishes a strong foundation for future enhancements involving advanced machine learning techniques, distributed event streaming technologies, and cloud-native deployments.

REFERENCES

1. FastAPI Documentation
2. React Documentation
3. SQLite Documentation
4. Ollama Documentation
5. Llama 3.1 Documentation
6. JWT Authentication Standards
7. Discord Webhook Documentation
8. Statistical Anomaly Detection Research Papers
9. Python Official Documentation
10. Docker Documentation

APPENDICES

Appendix A – API Endpoints

Appendix B – Database Schema

Appendix C – Test Cases

Appendix D – Dashboard Screenshots

Appendix E – Docker Configuration

Appendix F – User Guide